11 - NORMALIZAÇÃO E INDEXAÇÃO: CORRIGINDO O SMARTCOFFEE

Banco de Dados • 1FN • 2FN • 3FN • INDEX • EXPLAIN • Auditoria de Estrutura

Abertura

> **“Antes de otimizar um banco, precisamos ter certeza de que ele foi estruturado corretamente.”**

Situação-problema

O SmartCoffee cresceu rapidamente. Durante uma fase de testes, duas tabelas foram criadas de forma inadequada por uma equipe iniciante. Elas funcionam para armazenar alguns dados, mas apresentam repetição, dependências incorretas e risco de inconsistência.

Missão da turma: atuar como equipe de revisão técnica. Primeiro corrigir a modelagem usando normalização. Depois avaliar quais índices fazem sentido para as consultas do sistema.

1. Tabela problemática A - PEDIDO_COMPLETO_ERRADO

```
id_pedido
data_pedido
cliente_nome
cliente_email
cliente_cidade
produto1
produto2
produto3
categoria1
categoria2
quantidade1
quantidade2
quantidade3
```

Exemplo de registros

id	cliente	produto1	produto2	categoria1	categoria2
1	Ana Lima	Espresso	Brownie	Cafés	Doces
2	Ana Lima	Cappuccino	Cookie	Cafés	Doces

Problemas: dados do cliente repetidos; número fixo de produtos; grupos repetitivos; categoria repetida; dificuldade de atualização e expansão.

2. Descobrindo a 1FN pelo problema

A primeira correção deve eliminar grupos repetitivos como `produto1`, `produto2`, `produto3`.

```
PEDIDO
id_pedido
data_pedido
id_cliente

ITEM_PEDIDO
id_item
id_pedido
id_produto
quantidade
```

A 1FN faz sentido aqui porque um pedido pode possuir qualquer quantidade de itens sem criar novas colunas.

3. Separando CLIENTE, PRODUTO e CATEGORIA

Os dados do cliente não pertencem ao pedido. Eles descrevem o cliente. O nome da categoria também não deve ser repetido em cada produto.

```
CLIENTE
id_cliente PK
nome
email
cidade

CATEGORIA
id_categoria PK
nome

PRODUTO
id_produto PK
nome
id_categoria FK

PEDIDO
id_pedido PK
data_pedido
id_cliente FK

ITEM_PEDIDO
id_item PK
id_pedido FK
id_produto FK
quantidade
preco_unitario
```

4. Tabela problemática B - FORNECEDOR_PRODUTO_ERRADO

```
id_fornecedor PK
id_produto PK
nome_fornecedor
telefone_fornecedor
nome_produto
preco_atual_produto
preco_compra
prazo_entrega_dias
```

A chave é composta por `(id_fornecedor, id_produto)`.

Onde está o problema?

- `nome_fornecedor` depende apenas de `id_fornecedor`;
- `telefone_fornecedor` depende apenas de `id_fornecedor`;
- `nome_produto` depende apenas de `id_produto`;
- `preco_compra` depende da relação fornecedor-produto.

Aqui aparece a 2FN de forma concreta: em uma chave composta, atributos comuns devem depender da chave inteira.

5. Estrutura corrigida da 2FN

```
FORNECEDOR
id_fornecedor PK
nome
telefone

PRODUTO
id_produto PK
nome
preco

FORNECEDOR_PRODUTO
id_fornecedor PK/FK
id_produto PK/FK
preco_compra
prazo_entrega_dias
```

6. Aplicando a 3FN

Considere agora uma versão inadequada de PRODUTO:

```
PRODUTO
id_produto
nome_produto
id_categoria
nome_categoria
```

`nome_categoria` depende de `id_categoria`. Portanto, ele deve estar em CATEGORIA.

Resumo: 1FN resolve grupos repetitivos; 2FN remove dependências parciais; 3FN remove dependências indevidas entre atributos não-chave.

7. Antes e depois

Antes	Depois
Uma tabela grande misturando várias entidades.	Entidades separadas e relacionadas.
Dados duplicados.	Dados centralizados.
Atualização arriscada.	Alteração feita em um único lugar.
Produtos limitados a colunas fixas.	Itens ilimitados por relacionamento 1:N.

8. Redundância que deve permanecer

`ITEM_PEDIDO.preco_unitario` deve guardar o preço praticado no momento da venda.

> Mesmo que PRODUTO.preco mude amanhã, o histórico do pedido precisa continuar correto.

9. Agora sim: indexação

Depois que o modelo foi corrigido, podemos analisar consultas reais.

Consulta A - produto por nome

```
SELECT *
FROM produto
WHERE nome = 'Cappuccino';
```

Candidato:

```
CREATE INDEX idx_produto_nome
ON produto(nome);
```

Consulta B - pedidos de um cliente por data

```
SELECT *
FROM pedido
WHERE id_cliente = 5
  AND data_pedido >= '2026-10-01'
ORDER BY data_pedido;
```

Candidato:

```
CREATE INDEX idx_pedido_cliente_data
ON pedido(id_cliente, data_pedido);
```

Consulta C - itens de um pedido

```
SELECT *
FROM item_pedido
WHERE id_pedido = 20;
```

> Antes de criar um índice, verifique se a FK ou outra estrutura já gerou um índice adequado no SGBD.

10. INDEX não corrige tabela errada

MODELO ERRADO + INDEX = MODELO ERRADO MAIS RÁPIDO

MODELO CORRETO + INDEX JUSTIFICADO = ESTRUTURA CONSISTENTE E ACESSO EFICIENTE

11. SHOW INDEX e EXPLAIN

```
SHOW INDEX FROM pedido;

EXPLAIN
SELECT *
FROM pedido
WHERE id_cliente = 5
  AND data_pedido >= '2026-10-01';
```

A turma deve observar índices existentes e comparar o plano antes/depois da criação de um índice.

12. Roteiro da aula prática

1. Carregar as duas tabelas problemáticas.
2. Identificar redundâncias e anomalias.
3. Aplicar 1FN na tabela de pedidos.
4. Separar CLIENTE, PEDIDO, PRODUTO, CATEGORIA e ITEM_PEDIDO.
5. Aplicar 2FN em FORNECEDOR_PRODUTO_ERRADO.
6. Aplicar 3FN em PRODUTO/CATEGORIA.
7. Inserir dados na estrutura corrigida.
8. Executar consultas reais.
9. Usar SHOW INDEX.
10. Criar índices justificados.
11. Comparar usando EXPLAIN.

13. Atividade - Auditoria do SmartCoffee

1. Identifique pelo menos 5 problemas em PEDIDO_COMPLETO_ERRADO.

2. Explique qual problema está relacionado à 1FN.

3. Reestruture o cenário em CLIENTE, CATEGORIA, PRODUTO, PEDIDO e ITEM_PEDIDO.

4. Identifique dependências parciais em FORNECEDOR_PRODUTO_ERRADO.

5. Reestruture FORNECEDOR, PRODUTO e FORNECEDOR_PRODUTO.

6. Explique o problema de nome_categoria dentro de PRODUTO.

7. Justifique a permanência de preco_unitario em ITEM_PEDIDO.

8. Liste índices existentes em PRODUTO e PEDIDO.

9. Crie um índice para pesquisa por nome de produto.

10. Crie índice composto para pedidos por cliente/data.

11. Execute EXPLAIN antes e depois.

12. Explique por que não devemos indexar todas as colunas.

14. Checklist

- ☐ Consigo reconhecer uma tabela mal estruturada.
- ☐ Sei identificar grupos repetitivos.
- ☐ Entendo 1FN, 2FN e 3FN usando o SmartCoffee.

- ☐ Sei diferenciar redundância ruim de histórico necessário.
- ☐ Sei identificar candidatos a índice a partir de consultas.
- ☐ Sei usar SHOW INDEX.
- ☐ Sei criar índice simples e composto.
- ☐ Sei usar EXPLAIN de forma introdutória.

15. Mapa mental

SMARTCOFFEE COM PROBLEMAS
↓
IDENTIFICAR REDUNDÂNCIAS
↓
1FN
↓
2FN
↓
3FN
↓
MODELO CORRIGIDO
↓
CONSULTAS REAIS
↓
SHOW INDEX
↓
CREATE INDEX
↓
EXPLAIN
↓
BANCO MAIS CONSISTENTE E EFICIENTE

Encerramento

Primeiro corrigimos o significado e a estrutura dos dados.
Depois pensamos em velocidade.